

Model Context Protocol (MCP) for Ansible Automation Platform

Version 1, 2026-09-25

Welcome

Introduction

Welcome to the learning section of this course. In this chapter, you will understand what the Model Context Protocol is, how it works, and how Ansible Automation Platform uses it to connect with AI tools.

We'll break this down into bite-sized sections that follow a simple pattern: **What is it?** → **Why is it needed?** → **How does it work?**

What You'll Learn

By the end of this chapter, you will be able to:

- Define the Model Context Protocol and understand its role in the AI ecosystem
- Understand the architecture of MCP and how different components interact
- Explain Ansible Automation Platform's MCP server and its capabilities
- Describe the complete communication workflow from user to AI to AAP
- Identify and describe the 6 AAP MCP toolsets
- Understand the end-to-end workflow for deploying and using MCP

Prerequisites

This course assumes that you have the following prior experience:

- Ansible Automation Platform fundamentals:
- What is AI and how to Prompt to an AI tool.

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Andrew Block (Subject Matter Expert)
- Amrinder Singh (Content Architect)
- Anna Swicegood (Learning Product Manager)

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Model Context Protocol (MCP) for Ansible Automation Platform

Welcome to the learning section of this course. In this chapter, you will understand what the Model Context Protocol is, how it works, and how Ansible Automation Platform uses it to connect with AI tools.

We'll break this down into bite-sized sections that follow a simple pattern: **What is it?** → **Why is it needed?** → **How does it work?**

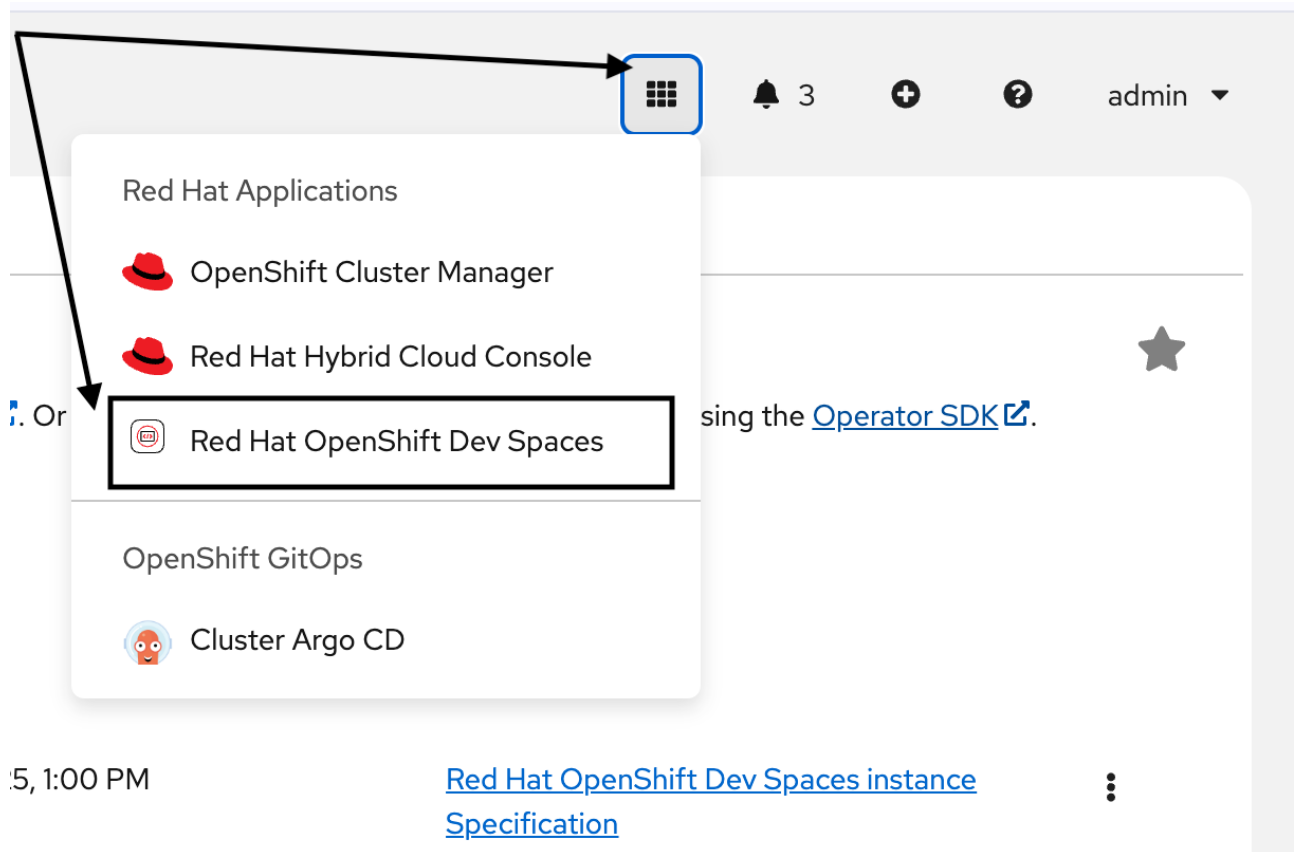
What You'll Learn

By the end of this chapter, you will be able to:

- Define the Model Context Protocol and understand its role in the AI ecosystem
- Understand the architecture of MCP and how different components interact
- Explain Ansible Automation Platform's MCP server and its capabilities
- Describe the complete communication workflow from user to AI to AAP
- Identify and describe the 6 AAP MCP toolsets
- Understand the end-to-end workflow for deploying and using MCP

1: Launching Dev Spaces from OpenShift

1. Launch the [OpenShift console](#) and login using the credentials provided in the RHDP login page.
2. Select the **htpasswd_provider** button and use the credentials provided in the RHDP login page to login to the OpenShift console use the URL.
3. Use the 9-block icon in the upper right to launch *Red Hat OpenShift Dev Spaces* directly from the OpenShift console header.

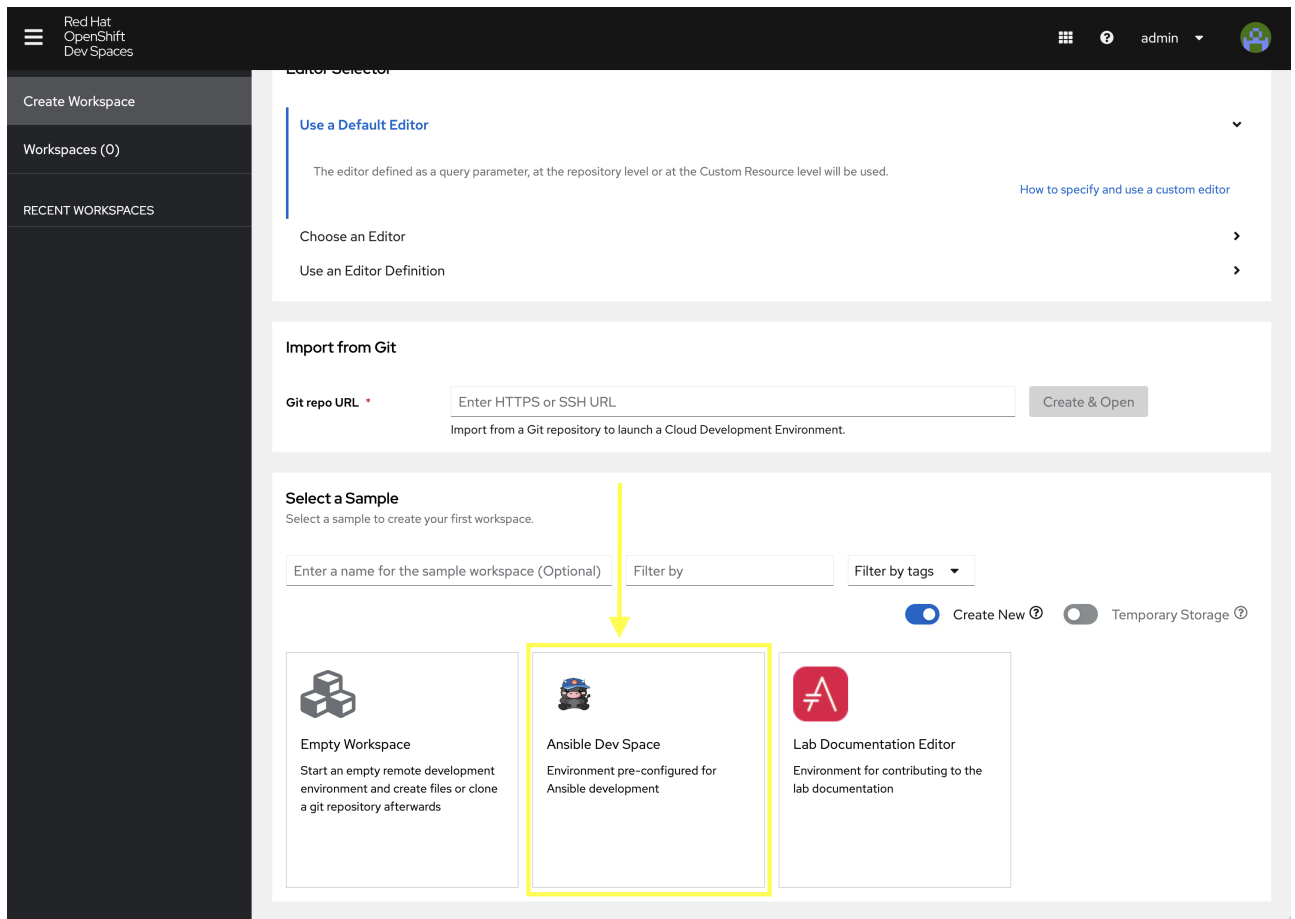


== Dev Spaces Workspace

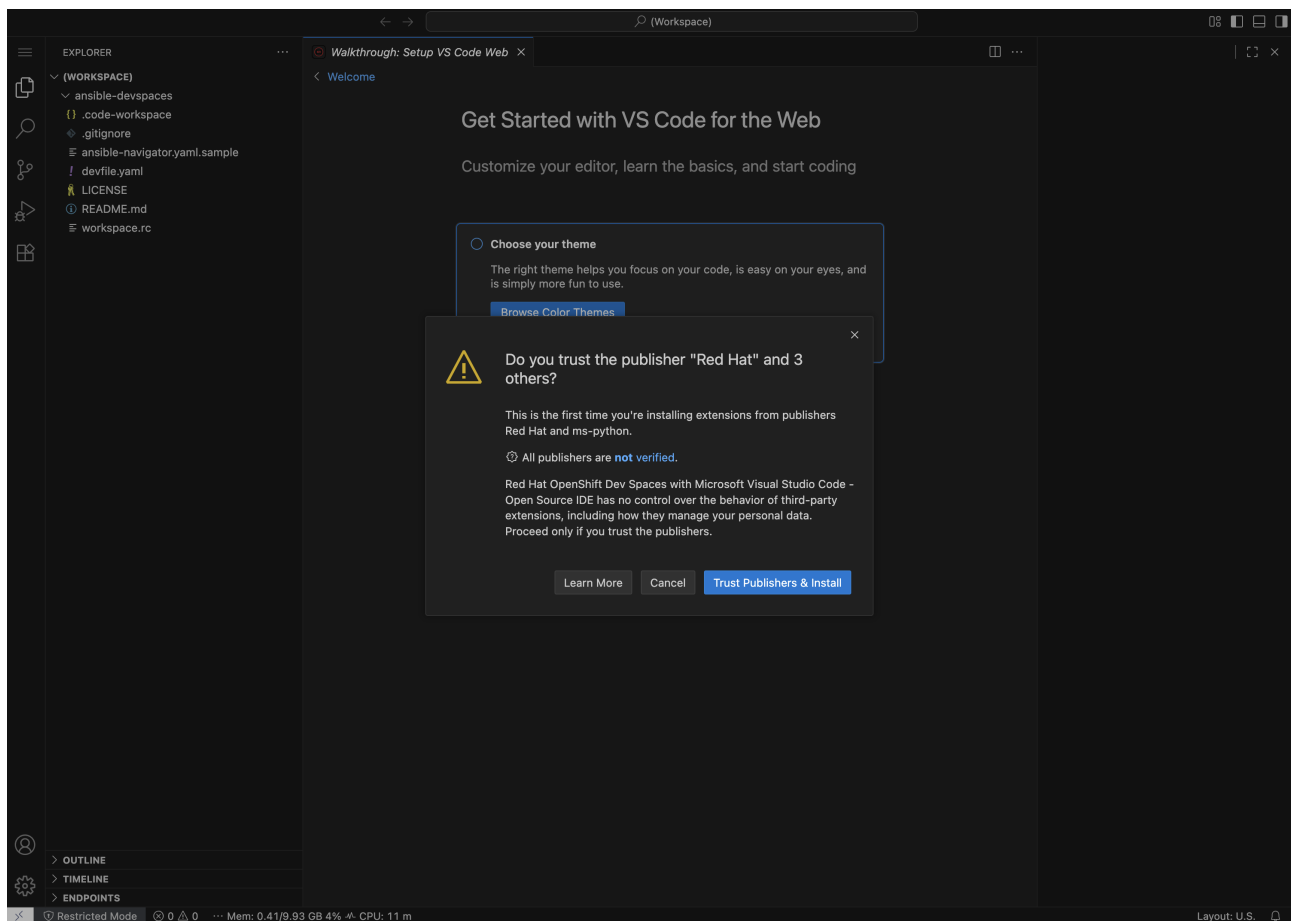
Launching a specific Dev Spaces Workspace

Once you have launched the OpenShift Dev Spaces console, use the following steps to access your Ansible Dev Space workspace.

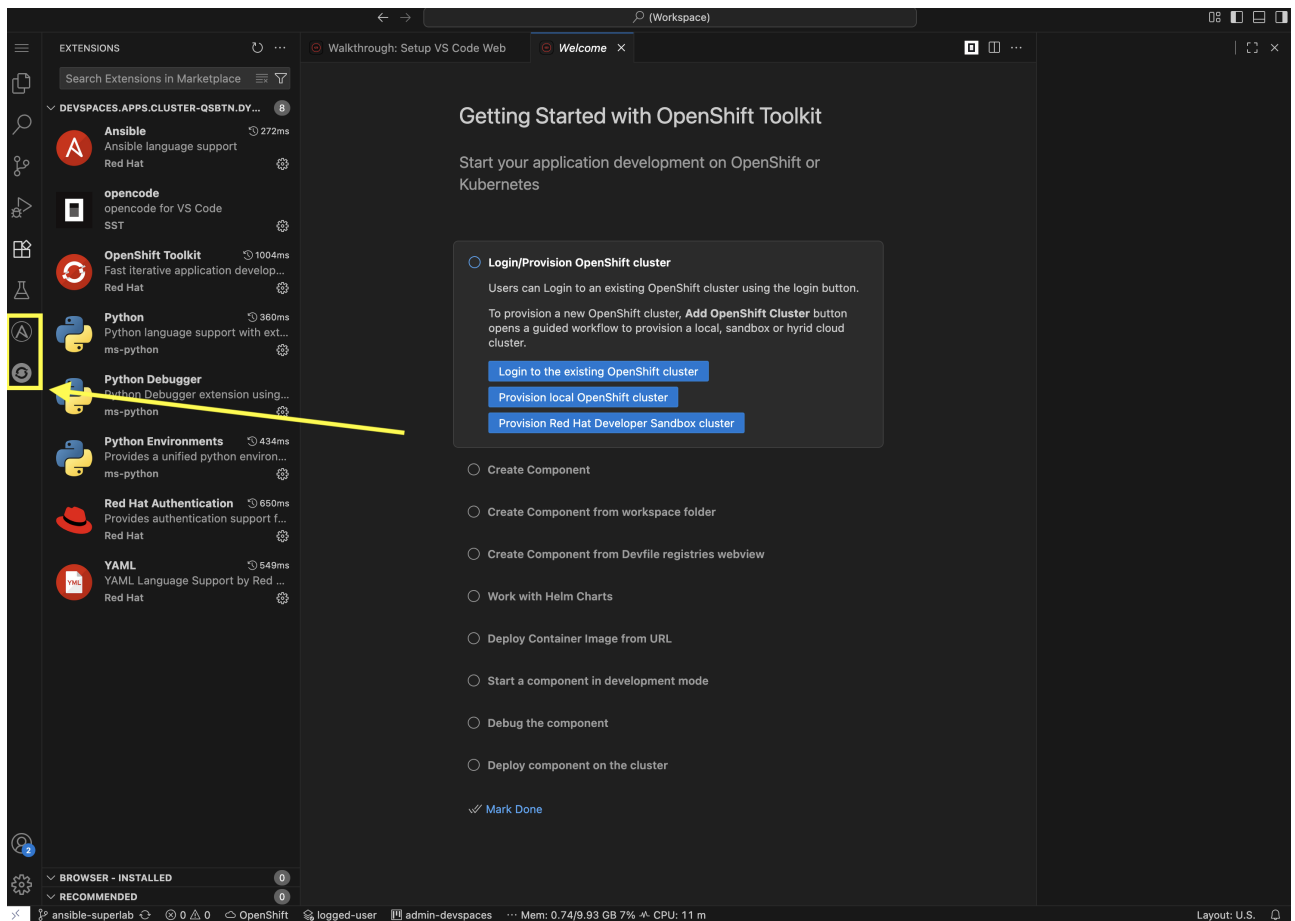
1. Login via OpenShift authentication using the same credentials as the OpenShift console
2. Select **Allow selected permissions** to authorize access and launch the console
3. Scroll down and select the workspace sample titled **Ansible Dev Space**



It will take a few minutes for the Dev Spaces instance to initialize. When it has loaded, a Visual Studio Code interface will appear.



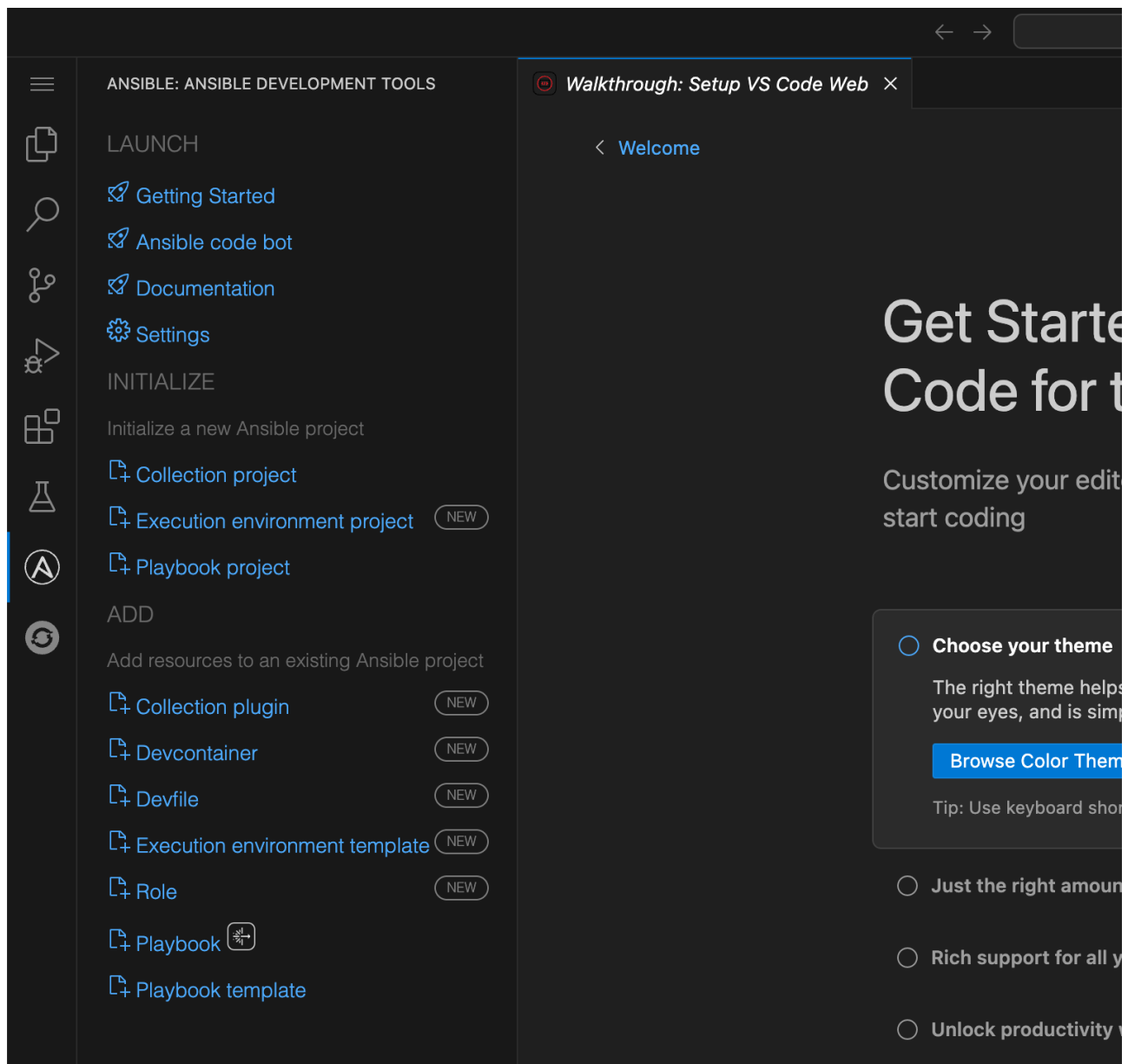
4. Click the **Trust Publishers and Install** button to trust the extension publishers that are included within the Dev Spaces workspace
5. Click the **Yes, I trust the Authors** button to trust the workspace authors (your instructors).
6. Wait a few minutes until the additional extension icons appear for Ansible and OpenShift on the left side.



Ansible and OpenShift Extensions

For this lab, we have provided a pre-configured configuration that included all the extensions along with additional configurations to curate your development environment.

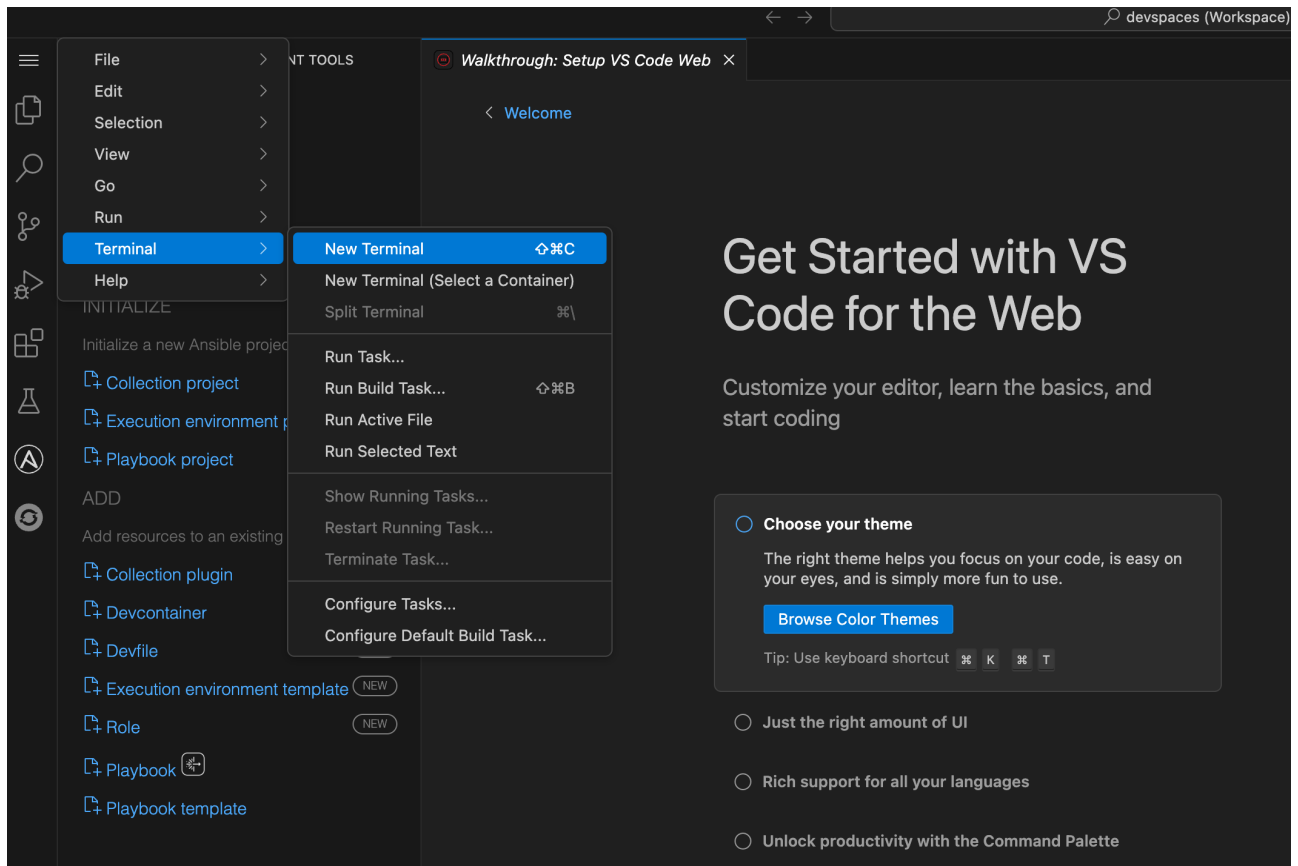
1. Click on the **Ansible** extension icon to access the Ansible Development Tools extension (covered in a subsequent lab)



Accessing a Terminal

The majority of the exercises in this lab will be performed using the Visual Studio Code terminal.

1. Open a new terminal by selecting the VS Code menu starting with the hamburger (3 horizontal lines) in the top left, then selecting **Terminal** → **New Terminal**



2. Explore the environment:

```
whoami
```

```
user
```

3. Check the Dev Spaces Workspace container OS release:

```
cat /etc/redhat-release
```

```
Red Hat Enterprise Linux release 9.7 (Plow)
```

4. This workspace provides access to **ansible** and additional developer tools:

```
ansible --version
```

```
ansible [core 2.19.3]
  config file = None
  configured module search path = ['/home/user/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.11/site-packages/ansible
```

```
ansible collection location =  
/home/user/.ansible/collections:/usr/share/ansible/collections  
executable location = /usr/local/bin/ansible  
python version = 3.11.13 (main, Aug 21 2025, 00:00:00) [GCC 11.5.0 20240719 (Red  
Hat 11.5.0-11)] (/usr/bin/python3.11)  
jinja version = 3.1.6  
pyyaml version = 6.0.3 (with libyaml v0.2.5)
```

Ready? Start with Section 1 below!

What is MCP?

What is the Model Context Protocol?

The **Model Context Protocol (MCP)** is an open-source standard that provides a universal way for AI models to interact with external data and tools.

Think of it as "**USB-C for the AI ecosystem**" — just like USB-C works with many different devices without custom adapters, MCP works with many different AI models and tools without requiring custom integrations.

[MCP as USB-C for AI] | *mcp-concept.png*

Instead of building a unique integration every time you want an AI model to interact with a new system, MCP provides a standardized method. This means:

- AI models can connect to different data sources without model-specific code
- You can swap AI providers without rewriting integrations
- Security and permissions are handled consistently

Why is MCP Needed?

Imagine you want to use ChatGPT to query your servers, while a colleague uses Claude to do the same thing. Without a standard protocol:

- ChatGPT integration team writes custom code to connect ChatGPT to your servers
- Claude integration team writes completely different code to do the same thing
- You have duplicated code, duplicated bugs, and duplicated maintenance

With MCP, both ChatGPT and Claude connect to the same MCP server. No duplication. No redundancy.

The Real Problem MCP Solves

AI models like Claude or ChatGPT are trained on data **frozen at a point in time**. They don't know:

- What jobs ran in your Ansible Automation Platform **yesterday**

- Which servers are currently online in your inventory
- What security events happened **today**
- Real-time metrics or current status

MCP solves this by giving AI models **real-time access** to your live systems. They can ask questions and get current answers, not guesses based on training data.

How Does It Work? (Simple Version)

MCP works in three simple steps:

MCP Flow

1. **You ask** → You send a natural language request to your AI tool
2. **AI translates** → The AI understands your intent and calls the appropriate MCP tool
3. **MCP delivers** → The MCP server fetches real data and returns it to the AI

Example

```
You: "List my recent Ansible jobs"
  []
AI: "I need to call the job_list tool with recent=true"
  []
MCP Server: Queries AAP and returns the list of jobs
  []
AI: "Here are your last 5 jobs: ..."
```

Next Steps

You understand **what** MCP is. Now let's understand **how** it's built — the architecture and components.

→ **Continue to Section 2: MCP Architecture**

MCP Architecture

What are the Building Blocks?

MCP is built from **four main components**. Let's break them down:

Component 1: The AI Client (The Requester)

This is where **you** interact with the AI. It could be:

- **Claude Desktop** — A desktop application by Anthropic
- **Cursor IDE** — A code editor with AI built in
- **LibreChat** — A web-based chat interface

- **ChatGPT** — OpenAI's chatbot
- **Any MCP-compatible tool** you're using

Role: You enter natural language requests here. "Show me failed jobs from yesterday." The client sends your request to the AI model.

Component 2: The AI Model (The Translator)

This is the **large language model (LLM)** — the brain of the operation.

Examples: * Claude (Anthropic) * GPT-4 (OpenAI) * Qwen (Alibaba) * Any LLM that supports MCP

Role: It interprets your natural language request and figures out which MCP **tools** to call and with what parameters.

Example Interpretation

Your Input: "How many Windows servers do I have?"

AI Model Thinks: "The user wants server counts. I should:

1. Call `inventory_list` to get hosts
2. Filter for hosts with `'os_windows'` group
3. Count them"

Component 3: MCP Server (The Gatekeeper)

The **MCP Server** is the bridge between the AI and your infrastructure.

Role: * Receives structured requests from the AI model * Validates the request using your API token
 * Executes the request against your system (AAP, in this case) * Standardizes the response and sends it back to the AI

The MCP server **never trusts the AI directly**. It requires authentication and applies your system's security policies.

Component 4: Your System (The Executor)

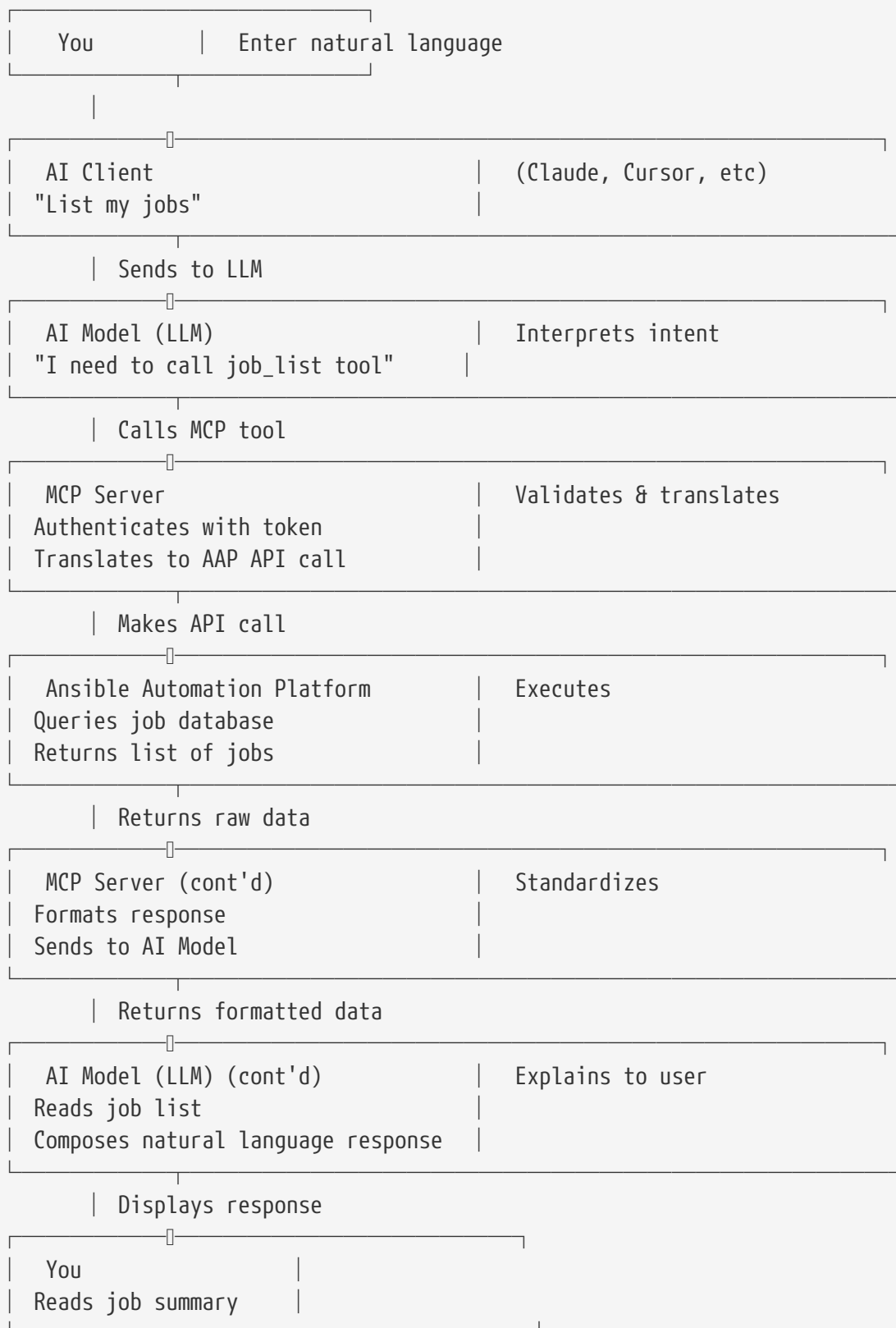
In this course, it's **Ansible Automation Platform (AAP)**, but it could be:

- Your cloud platform (AWS, Azure, GCP)
- Your internal database
- Your monitoring system
- Any system with an API

Role: Executes the actual action or returns the requested data.

How They Connect

Here's the complete architecture:



What About Security?

The MCP Server is the **security checkpoint**:

1. **Authentication**: Uses your AAP API token to verify requests
2. **Authorization**: Only allows actions your user account has permission to perform

3. **Standardization:** Ensures all requests follow the same format
4. **Validation:** Checks that parameters are valid before sending to AAP

Security Example

AI asks: "Delete all hosts"

MCP Server checks:

- ✓ Is the API token valid? YES
- ✓ Does the user have delete permission? NO
- ✗ DENIED - Permission rejected

Response to AI: "You don't have permission to delete hosts"

Next Steps

You understand the components. Now let's see how **Ansible Automation Platform** uses MCP.

→ **Continue to Section 3: AAP Integration & Toolsets**

AAP Integration & Toolsets

What is the Ansible MCP Server?

The **Ansible MCP Server** is Ansible Automation Platform's implementation of the MCP standard.

It enables connecting AAP with any preferred external AI tool (such as Claude, Cursor, or ChatGPT). The AI tools can access information about your AAP environment and perform tasks using natural language.

Instead of logging into the AAP web UI manually, you can ask an AI: "Launch the 'Deploy Web Servers' job template" and it does it for you.

Why AAP Needs an MCP Server

AAP manages:

- **Thousands of servers and devices**
- **Complex automation workflows and job executions**
- **Sensitive credentials and access control**
- **User teams and role-based permissions**
- **Audit logs and compliance data**

Before MCP, the only way to query this was:

1. Log into the web UI manually
2. Click through menus

3. Run API commands with `curl` or write custom scripts

MCP enables: Ask the AI in natural language, get instant answers.

The 6 AAP MCP Toolsets

The Ansible MCP Server exposes **6 separate toolsets**, each focused on a different area of AAP:

1. Job Management

What it does: Execute and monitor automation jobs.

Tools you can call:

- List job templates
- Start a job run
- Check job status
- View job output/logs
- List recent jobs

Example queries:

```
"Show me failed jobs from the last 24 hours"
"Launch the 'Configure Servers' job template"
"What jobs are currently running?"
```

2. Inventory Management

What it does: Manage hosts, groups, and inventories.

Tools you can call:

- List inventories
- Get hosts in a group
- List inventory groups
- Get host details and variables
- Filter hosts by properties

Example queries:

```
"Show me all Windows hosts"
"Which hosts are in the 'production' group?"
"List hosts that haven't checked in for 7 days"
```

3. System Monitoring

What it does: Monitor platform health and performance metrics.

Tools you can call:

- Check AAP health status
- Get execution node metrics
- View job execution statistics
- Monitor capacity and usage
- Get system performance data

Example queries:

```
"What's the health status of my AAP instance?"  
"Show me job execution metrics for the past week"  
"What's the capacity utilization?"
```

4. User Management

What it does: Manage users, teams, and organizations.

Tools you can call:

- List users and teams
- Get team memberships
- View organization details
- Check user roles and permissions
- Manage RBAC assignments

Example queries:

```
"List all users in my organization"  
"Who are the members of the ops team?"  
"What permissions does user 'alice' have?"
```

5. Security & Compliance

What it does: Access security policies, credentials, and audit logs.

Tools you can call:

- View audit logs
- List credentials
- Check security settings

- Review compliance data
- Monitor access and changes

Example queries:

```
"Show me audit logs from the past 24 hours"
"What credentials are stored in AAP?"
"List all security-related activity"
```

6. Platform Configuration

What it does: Configure platform settings and resources.

Tools you can call:

- List projects and their status
- View execution environments
- Get platform settings
- Manage integrations
- Configure system properties

Example queries:

```
"List all projects and their SCM status"
"What execution environments are available?"
"Show me platform configuration settings"
```

One Server, Many Endpoints

All 6 toolsets are served by **one MCP server** with **6 separate endpoints**:

Toolset	Endpoint Path
Job Management	/job_management/mcp
Inventory Management	/inventory_management/mcp
System Monitoring	/system_monitoring/mcp
User Management	/user_management/mcp
Security & Compliance	/security_compliance/mcp
Platform Configuration	/platform_configuration/mcp

When you configure an AI tool, you connect it to **all 6 endpoints** so the AI has access to everything.

Role-Based Access Control (RBAC)

The MCP server respects your AAP permissions. If you create an API token for a user with "read-only" permissions, the AI can only read data—it cannot launch jobs or make changes.

RBAC Example

Token created with SCOPE: Read

Allowed:

- ✓ List jobs
- ✓ View job output
- ✓ List inventories
- ✓ View hosts

Denied:

- ✗ Launch a job
- ✗ Create users
- ✗ Delete credentials
- ✗ Modify settings

This means:

- **Admins** can use the AI to do anything they could do manually
- **Read-only users** can only query information
- **Team members** respect team-level permissions
- **Audit logs** track all AI-driven actions

Next Steps

You understand the toolsets. Now let's see the **complete communication workflow**—how a request flows from user to AI to AAP.

→ **Continue to Section 4: Communication Workflow**

Communication Workflow

The Full Journey: From User to AAP

Let's trace exactly what happens when you ask the AI a question. This is the **complete communication workflow**.

Step-by-Step Breakdown

Step 1: The User Initiates a Request

You: "Show me failed jobs from the past 7 days"

This is a natural language request sent to your AI tool (Claude, Cursor, ChatGPT, etc).

What's happening:

- The AI client receives your message
- It sends it to the AI model for interpretation

Step 2: The AI Model Interprets and Plans

```
AI Model (thinking):  
"The user wants:  
1. A list of jobs  
2. Filtered to only FAILED status  
3. From the last 7 days  
  
I need to call the job_list tool with:  
- status = "failed"  
- limit = 7 days  
"
```

What's happening:

- The AI model understands your intent
- It maps your request to available MCP tools
- It prepares the structured request with parameters

Step 3: The MCP Server Receives the Request

```
Request from AI:  
{  
  "method": "tools/call",  
  "params": {  
    "name": "job_list",  
    "arguments": {  
      "status": "failed",  
      "limit_days": 7  
    }  
  }  
}
```

What's happening:

- The MCP server receives a structured tool call
- It validates the request format
- It checks that the tool exists

Step 4: The MCP Server Authenticates

MCP Server checks:

- ✓ API token is valid? YES
- ✓ Token belongs to active user? YES
- ✓ User has read permission? YES

Status: AUTHENTICATED

What's happening:

- The MCP server extracts your API token from the request headers
- It verifies the token with AAP
- It loads your user's permissions and RBAC settings

Step 5: The MCP Server Translates and Calls AAP

MCP Server translates to AAP API call:

GET /api/v2/jobs/?status=failed&created__gte=2024-09-08

Headers:

Authorization: Token <your-aap-api-token>
Content-Type: application/json

What's happening:

- The MCP server converts the tool call into an AAP API request
- It uses your API token to authenticate with AAP
- It applies any RBAC restrictions from your user account

Step 6: AAP Executes the Query

AAP Database Query:

SELECT jobs WHERE status='failed' AND created > '2024-09-08'
RESULT: 3 failed jobs found

What's happening:

- AAP queries its database
- It applies your user's permission restrictions
- It only returns jobs you're allowed to see
- It returns the results as JSON

Step 7: The MCP Server Standardizes the Response

Raw AAP Response (200 JSON objects with 50 fields each):

```
{
  "count": 3,
  "results": [
    {
      "id": 456,
      "name": "Deploy Config",
      "status": "failed",
      "created": "2024-09-10T14:32:00Z",
      "started": "2024-09-10T14:32:05Z",
      "finished": "2024-09-10T14:45:00Z",
      ...50 more fields...
    },
    ...
  ]
}
```

MCP Server Standardizes:

```
{
  "jobs": [
    {
      "id": 456,
      "name": "Deploy Config",
      "status": "failed",
      "duration": "12 minutes",
      "failure_reason": "Host unreachable"
    },
    ...
  ]
}
```

What's happening:

- The MCP server receives raw AAP data
- It extracts only relevant fields
- It formats data in a way the AI can easily understand
- It simplifies complex structures

Step 8: The AI Model Processes the Data

AI Model receives standardized response

AI Model thinks:

"I have 3 failed jobs. Let me summarize them for the user in a clear, readable format"

What's happening:

- The AI receives the structured data
- It reads and understands the job details
- It composes a human-readable summary

Step 9: The AI Client Displays the Response

You see in your AI chat:

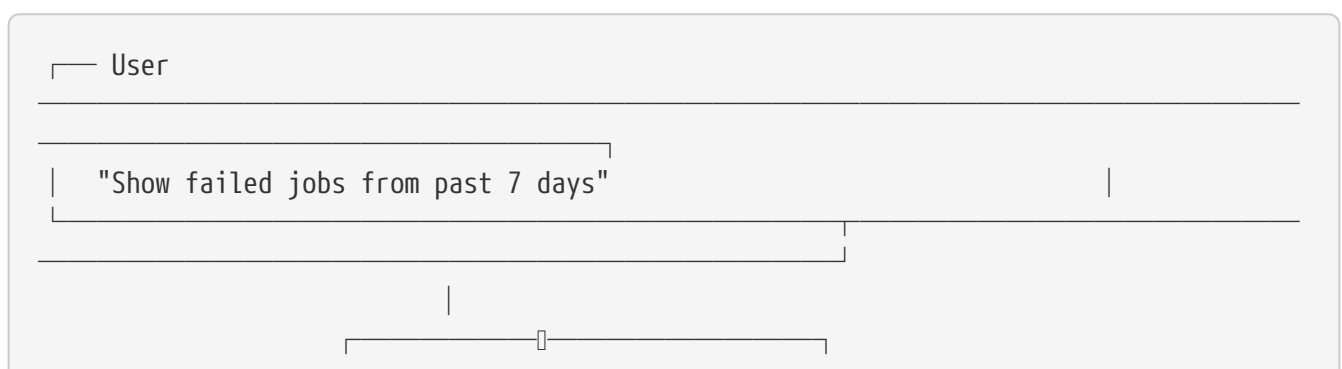
"You have 3 failed jobs from the past 7 days:

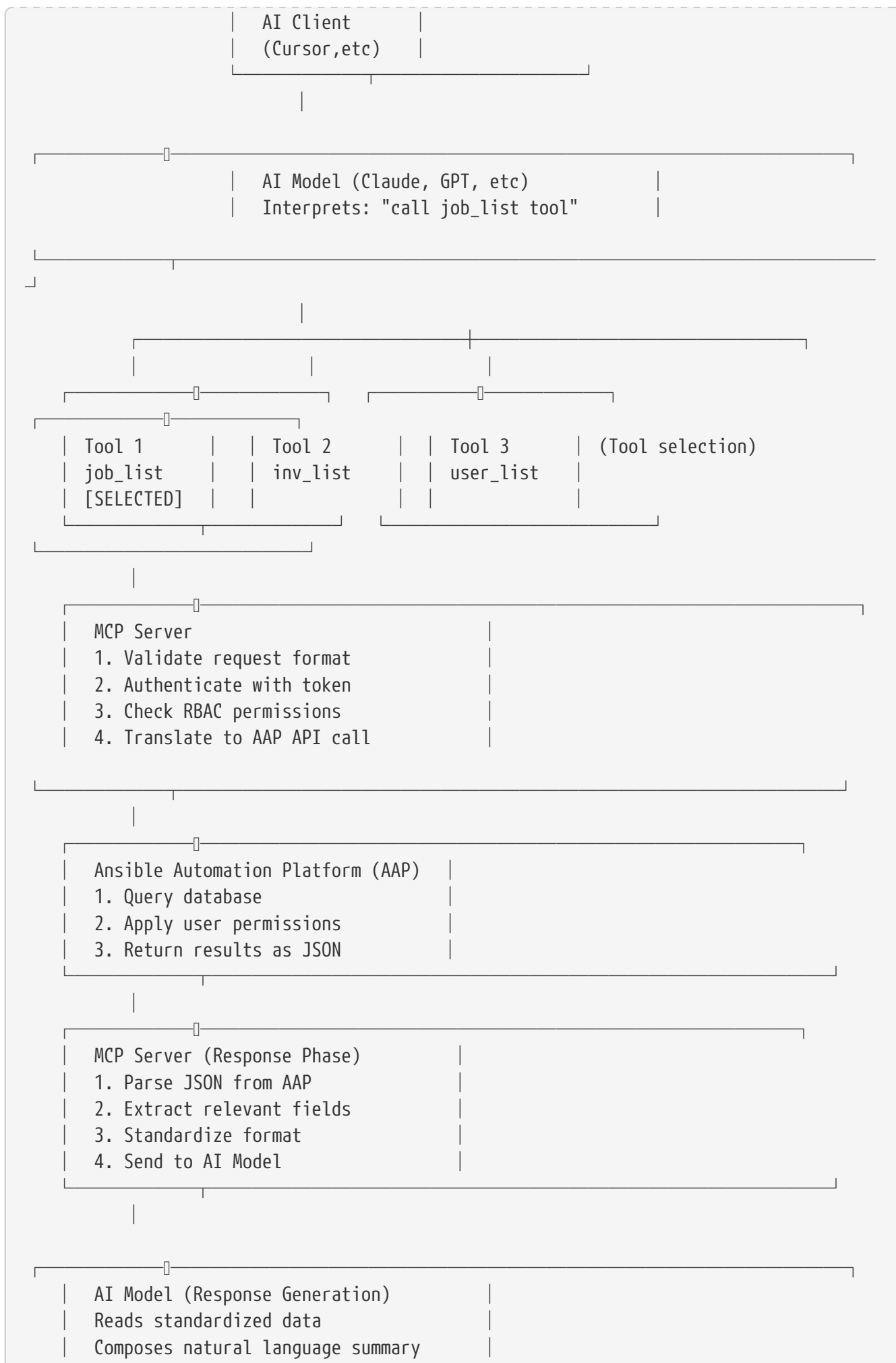
1. Deploy Config (Job #456)
 - Duration: 12 minutes
 - Status: FAILED
 - Reason: Host unreachable
 - Started: Sept 10, 2:32 PM
 2. Database Migration (Job #457)
 - Duration: 25 minutes
 - Status: FAILED
 - Reason: Permission denied
 - Started: Sept 11, 10:15 AM
 3. Cleanup Tasks (Job #458)
 - Duration: 5 minutes
 - Status: FAILED
 - Reason: Timeout
 - Started: Sept 12, 3:45 PM
- "

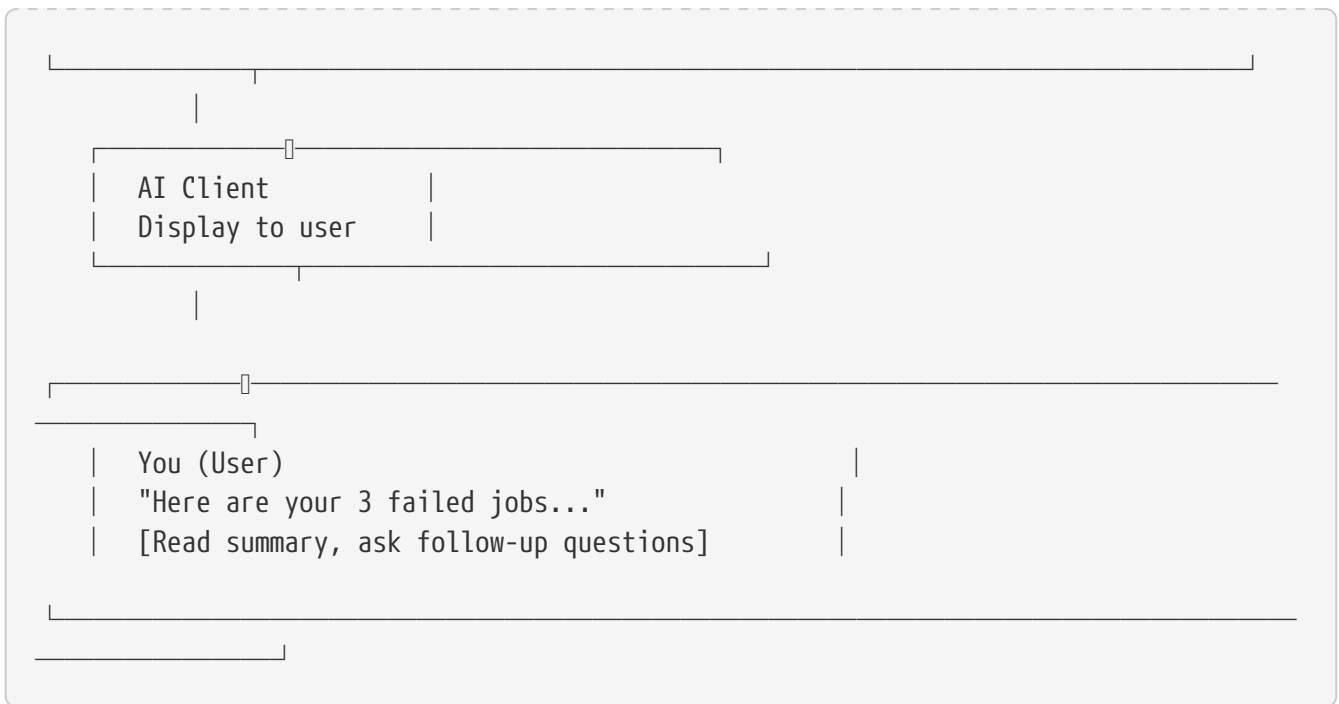
What's happening:

- The AI client displays the formatted response
- You read the summary
- You can now ask follow-up questions

Visualized Workflow







Key Insights

Security Checkpoints

The workflow has **multiple security checkpoints**:

1. ✓ Token validation (is the request from a valid user?)
2. ✓ RBAC check (does the user have permission for this action?)
3. ✓ Data filtering (only show data the user is allowed to see)

Data Flow

Data flows in **two directions**:

1. **Request direction**: Natural language → Structured tool call → API call → Database query
2. **Response direction**: Database result → Formatted response → Natural language summary → Display

No Direct AI-to-AAP Access

Important: The AI model never talks directly to AAP.

The MCP server is always in the middle:

- ✓ The AI cannot bypass authentication
- ✓ The AI cannot exceed user permissions
- ✓ All requests are logged and auditable
- ✓ All data is standardized and validated

Next Steps

You now understand the complete workflow. Let's look at what happens in real deployments.

→ **Continue to Section 5: End-to-End Workflow**

End-to-End Workflow

Getting MCP Running: The Three Steps

Now that you understand **what** MCP is and **how** it works, here's what you actually **do** to make it work and connect your external AI tool to Ansible Automation Platform (AAP) through the Ansible MCP Server. The process is broken down into three steps:

End to End Workflow

To make use of the Ansible MCP Server, the following steps are required:

1. Deploy the Ansible MCP Server within Ansible Automation Platform
2. Generate an API token for use by the external AI tool
3. Configure the external AI tool to connect to the Ansible MCP Server using the generated API token

Step 1: Deploy the MCP Server

Where Does It Run?

The Ansible MCP Server runs as a containerized service on your infrastructure. You can deploy it on:

- **OpenShift Container Platform (OCP)** — Most common for enterprise
- **RHEL with containers** — Simpler, on a single machine
- **Kubernetes** — Any Kubernetes cluster

Deploy the Ansible MCP Server to your OpenShift environment by enabling the MCP capability within the **AnsibleAutomationPlatform** custom resource.

1. Launch the [OpenShift Web Console](#)
2. Select the **htpasswd_provider** button and use the credentials provided in the RHDP login page to the OpenShift console.
3. Navigate to **Ecosystem** → **Installed Operators**
4. From the *Project* dropdown, ensure **aap** is selected
5. The **Ansible Automation Platform** operator is listed and installed within this project.

Red Hat OpenShift

Project: aap

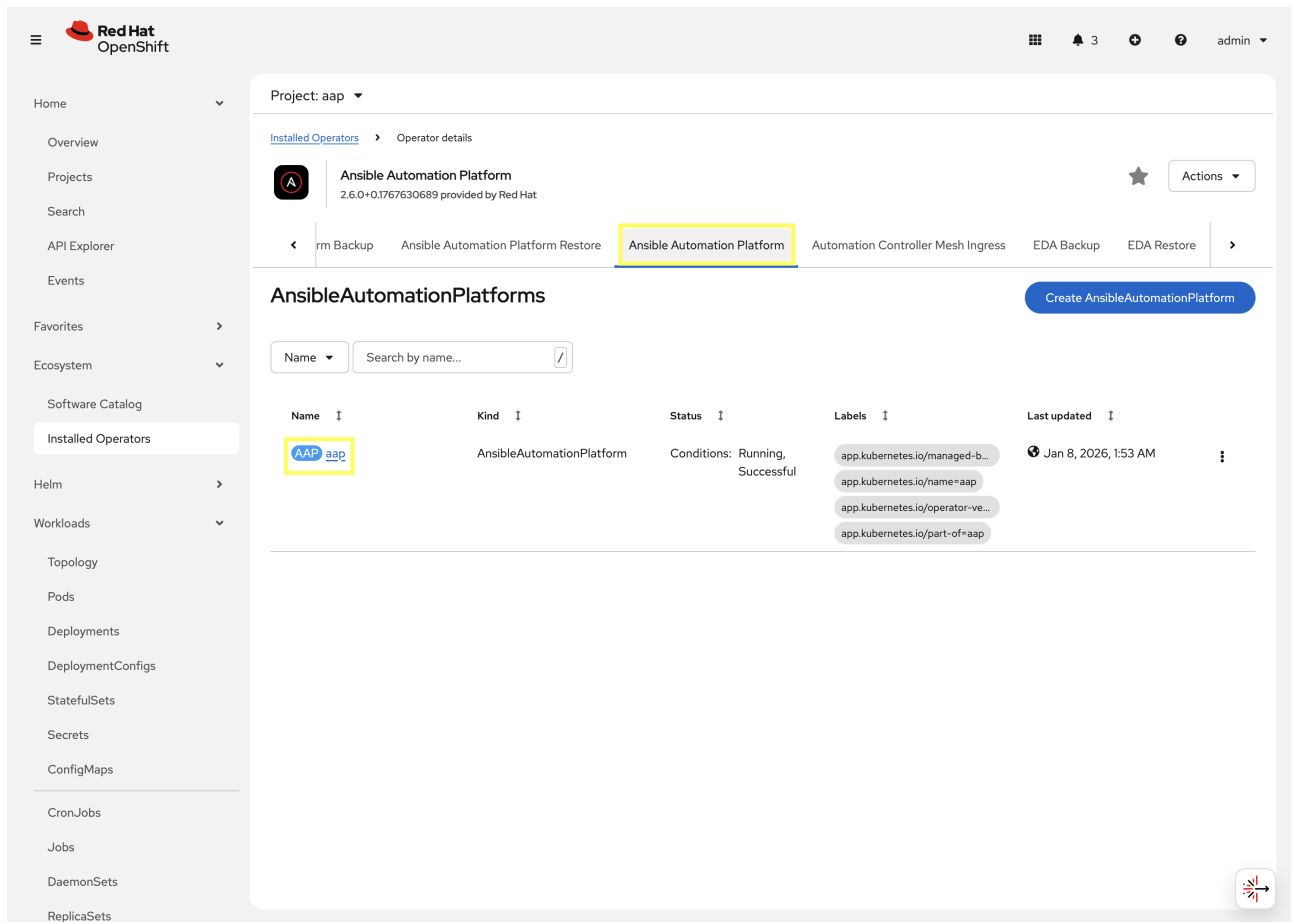
Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

Name Search by name...

Name	Managed Namespaces	Status	Last updated	Provided APIs
 Ansible Automation Platform 2.6.0-0.1767630689 provided by Red Hat	 NS aap	 Succeeded Up to date	 Jan 8, 2026, 2:32 PM	Automation Controller Backup Automation Controller Restore Automation Controller Automation Hub View 20 more...
 Streams for Apache Kafka 3.1.0-6 provided by Red Hat	 NS aap The operator is running in openshift-operators but is managing this namespace	 Succeeded Up to date	 Jan 8, 2026, 2:12 AM	Kafka Kafka Node Pool Kafka Connect Kafka Bridge View 5 more...
 Red Hat OpenShift Dev Spaces 3.25.0 provided by Red Hat	 NS aap The operator is running in devspaces but is managing this namespace	 Succeeded Up to date	 Jan 8, 2026, 2:10 AM	Red Hat OpenShift Dev Spaces instance Specification
 DevWorkspace Operator 0.38.0 provided by Devfile	 NS aap The operator is running in devspaces but is managing this namespace	 Succeeded Up to date	 Jan 8, 2026, 2:09 AM	DevWorkspace DevWorkspaceTemplate DevWorkspaceOperatorConfig
 Gitea Operator 2.1.0 provided by Red Hat Portfolio Technical Marketing and Platforms	 NS aap The operator is running in gitea-operator but is managing this namespace	 Succeeded Up to date	 Jan 8, 2026, 1:49 AM	Gitea
 Red Hat OpenShift GitOps	 NS aap The operator is running in openshift-gitops-operators	 Succeeded Up to date	 Jan 8, 2026, 1:48 AM	Argo CD NamespaceManagement

- Click on the **Ansible Automation Platform** operator which will display all of the resources that are managed by the operator.
- In the top tab, click on **Ansible Automation Platform** and select **aap**.



8. Click on the **YAML** tab to edit the resource in YAML format.

9. Add the following to the **spec** section by updating the configuration to enable the MCP server:

```
...
spec:
  mcp:
    disabled: false
    allow_write_operations: false
```



Setting **allow_write_operations** to **false** will restrict the MCP server to read-only operations. This is the preferred option, especially in Enterprise environments, when beginning to incorporate AAP with external AI tools. The AI tool may perform unwanted actions if write operations are enabled without fully understanding the implications. For the purpose of this lab, access to write operations will be restricted.

10. Click the **Save** button to apply the changes.

Once the configuration has been applied, the Ansible Automation Platform operator will begin to reconcile the changes. This process can take several minutes to complete.

The OpenShift Operator will then:

1. Create a new Pod for the MCP server

2. Expose it with a network route
3. Handle certificates and networking

How Long Does It Take?

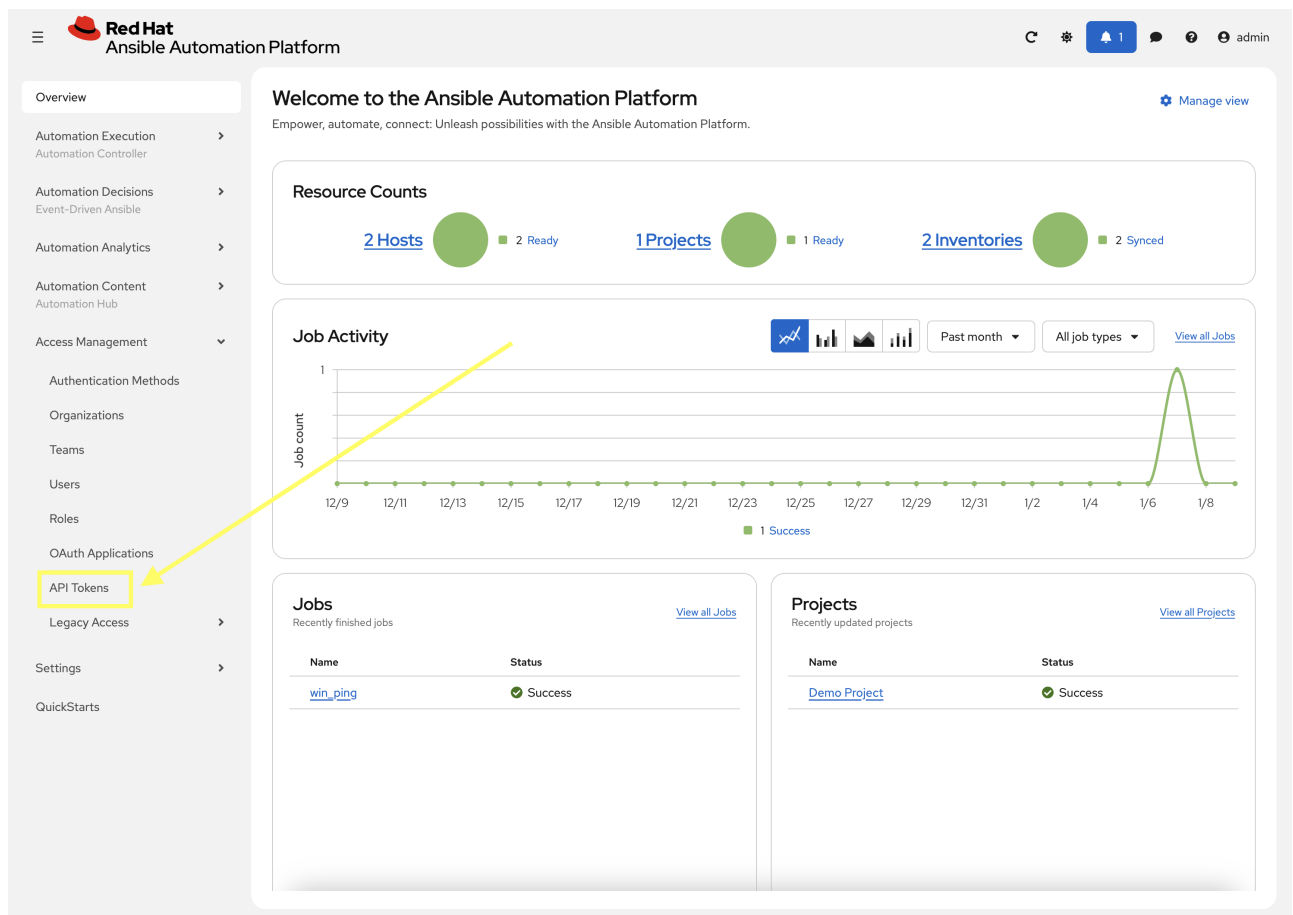
Usually **5-15 minutes** for the operator to reconcile and the MCP server to be ready.

Step 2: Generate an API Token

What's an API Token?

An **API token** is like a password, Create a Personal Access Token (PAT) by performing the following:

1. Launch the [Ansible Automation Platform](#) web interface and login using the credentials from the [Environment Details,window=_blank](#) page.
2. From the navigation menu on the left, expand **Access Management** and click on **API Token**.



3. On the API Token page, click the **Create API Token** button
4. On the Create API Token page, enter the following details:
 - **Description:** `mcp`
 - **OAuth Application:** Leave this field blank which will result in a Personal Access Token (PAT) being created.
 - **Scope:** Select **Read** to restrict the token to read-only operations.

The screenshot shows the 'Create API Token' interface in the Red Hat Ansible Automation Platform. The left sidebar contains navigation links: Overview, Automation Execution (Automation Controller), Automation Decisions (Event-Driven Ansible), Automation Analytics, Automation Content (Automation Hub), Access Management (Authentication Methods, Organizations, Teams, Users, Roles, OAuth Applications), API Tokens (selected), Legacy Access, Settings, and QuickStarts. The main panel is titled 'Create API Token' and includes a 'Description' field containing 'mcp', an 'OAuth application' dropdown menu, and a 'Scope' dropdown menu set to 'Read'. At the bottom, there are 'Create token' and 'Cancel' buttons.

5. Click **Create token** to generate the API token which will be displayed in the resulting dialog



Make sure to store the token securely, as it will not be displayed again.

3: Configuring External AI Tools

Now that the Ansible MCP Server is deployed and an API token has been generated, the final step is to configure external AI tools that can be used to connect to the Ansible MCP Server using the generated API token. While there are multiple external AI tools that can be used to demonstrate the integration with the Ansible MCP Server, for the purpose of this lab, the following solutions will be demonstrated:

1. [OpenCode](#) - Terminal based AI agent that can be executed from within the OpenShift Dev Spaces workspace.
2. [LibreChat](#) - Web based AI chat tool that can be accessed via a web browser.

3.1: Using OpenCode with the Ansible MCP Server

[OpenCode](#) is an Open Source, terminal based AI agent. It is already installed in your Dev Spaces workspace and requires that a few steps be completed prior to its use.

3.1.1: Customizing a Dev Spaces Workspace

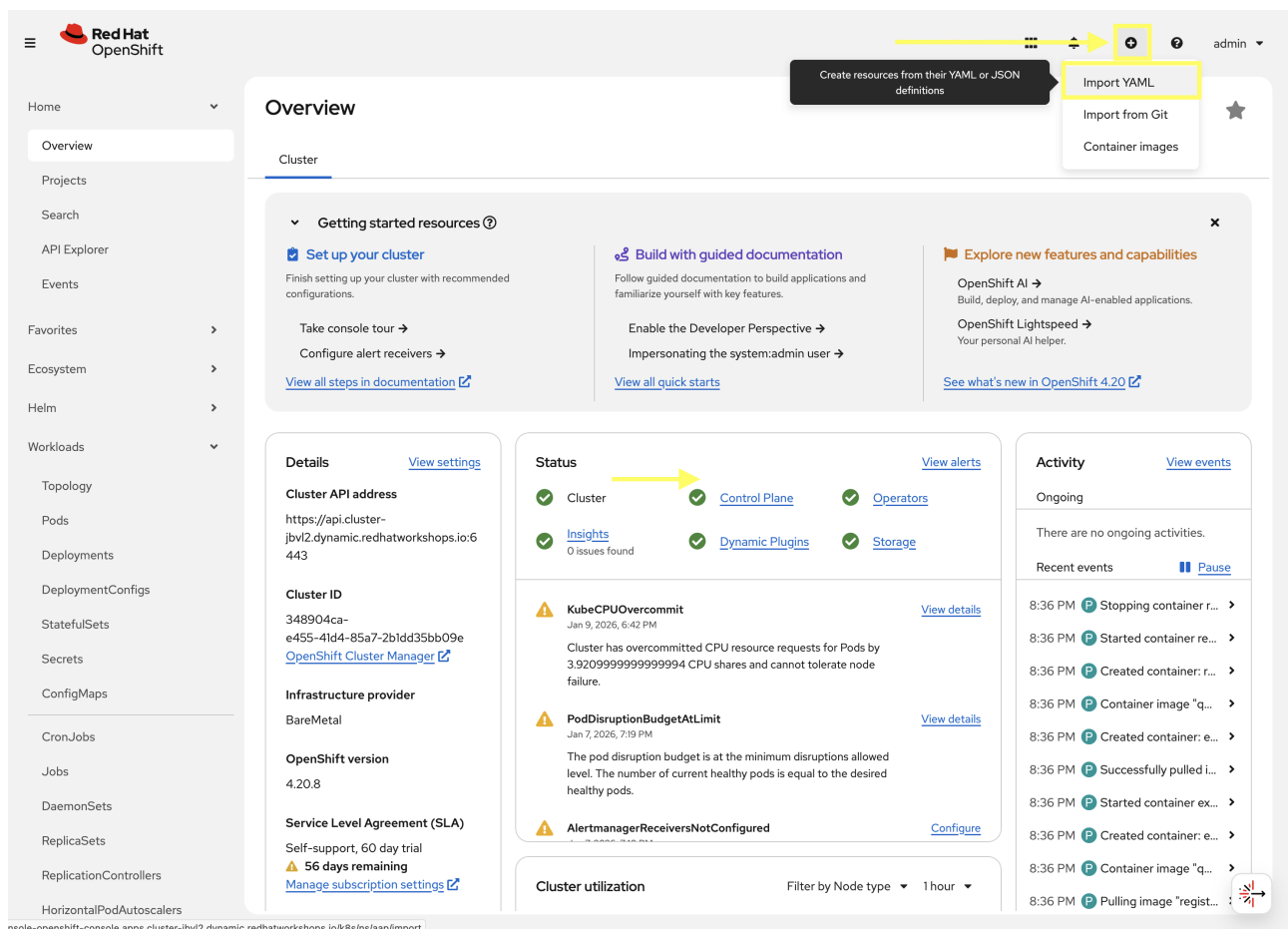
OpenShift Dev Spaces includes several different methods for configuring and customizing a workspace. For this section, we will create a `ConfigMap` and `Secret` resource within the `admin-devspaces` project containing your workspace to externalize and inject environment variables into

your workspace. These values will be referenced by the OpenCode configuration file that you will then define to drive the functionality of the tool.

Labels and annotations are applied to the **ConfigMap** and **Secret** resources so that they become eligible for injection into the Dev Spaces workspace. By setting the **controller.devfile.io/mount-to-devworkspace** label to **true**, the Dev Spaces controller will automatically mount the resources into the workspace. The contents of the **ConfigMap** or **Secret** can either be mounted as files or environment variables. This is dictated by the **controller.devfile.io/mount-as** annotation.

First, create a **ConfigMap** resource to contain the non-sensitive configuration values by performing the following in the OpenShift web console. Instead of using the OpenShift web console to interactively create the ConfigMap, you will import the raw YAML definition using the **Import YAML** feature.

1. Launch the **OpenShift Web Console**
2. Select the **htpasswd_provider** button and use the credentials provided in the **Environment Details,window=_blank** page to login to the OpenShift console
3. On the top navigation bar, click on the + icon and select **Import YAML**



4. Enter the following into the **Import YAML** dialog:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: superlab-devspaces-config
```

```

namespace: admin-devspaces
labels:
  controller.devfile.io/mount-to-devworkspace: 'true'
  controller.devfile.io/watch-configmap: 'true'
annotations:
  controller.devfile.io/mount-as: env
data:
  AAP_MCP_URL: https://aap-mcp-aap.{openshift_cluster_ingress_domain}/mcp
  LITELLM_MODEL_NAME: {litellm_available_models_list}
  LITELLM_URL: {litellm_api_base_url}
  OPENCODE_CONFIG: /projects/ansible-devspaces/opencode.json

```



Notice the value of the `controller.devfile.io/mount-as` annotation in the snippet above is set to `env`, which means the contents of the `ConfigMap` will be mounted as environment variables in the Dev Spaces workspace.

5. Click the **Create** button to create the `ConfigMap` resource.

Now, create a `Secret` resource to contain the sensitive configuration values by repeating the same steps as above by using the *Import YAML* feature.

1. On the top navigation bar, click on the + icon and select **Import YAML**
2. Enter the following into the *Import YAML* dialog. Make sure to replace `PLACEHOLDER_REPLACE_ME` with the AAP API Token you generated in the prior section:

```

kind: Secret
apiVersion: v1
metadata:
  name: superlab-devspaces-secret
  namespace: admin-devspaces
  labels:
    controller.devfile.io/mount-to-devworkspace: 'true'
    controller.devfile.io/watch-secret: 'true'
  annotations:
    controller.devfile.io/mount-as: env
stringData:
  AAP_MCP_API_KEY: PLACEHOLDER_REPLACE_ME
  LITELLM_API_KEY: {litellm_virtual_key}
type: Opaque

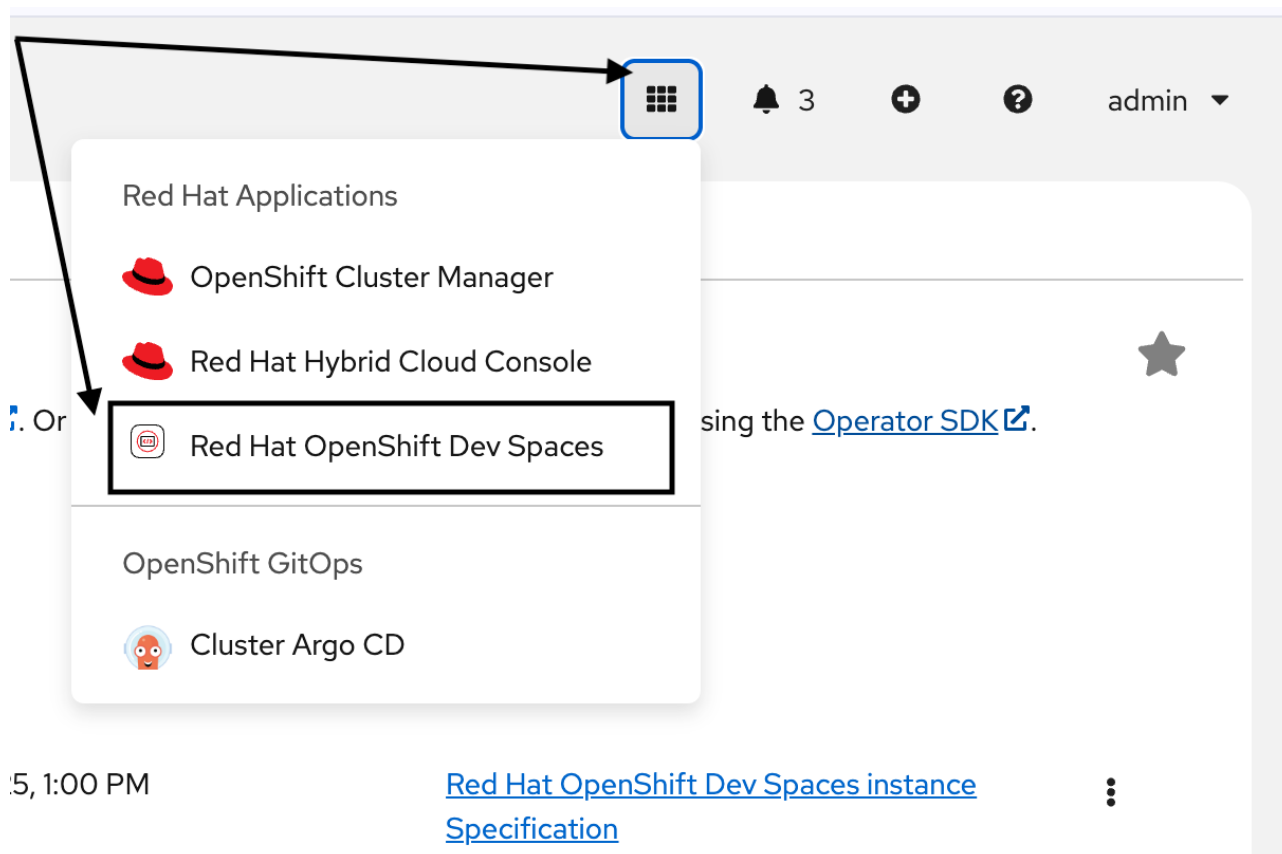
```

3. Click the **Create** button to create the `Secret` resource.

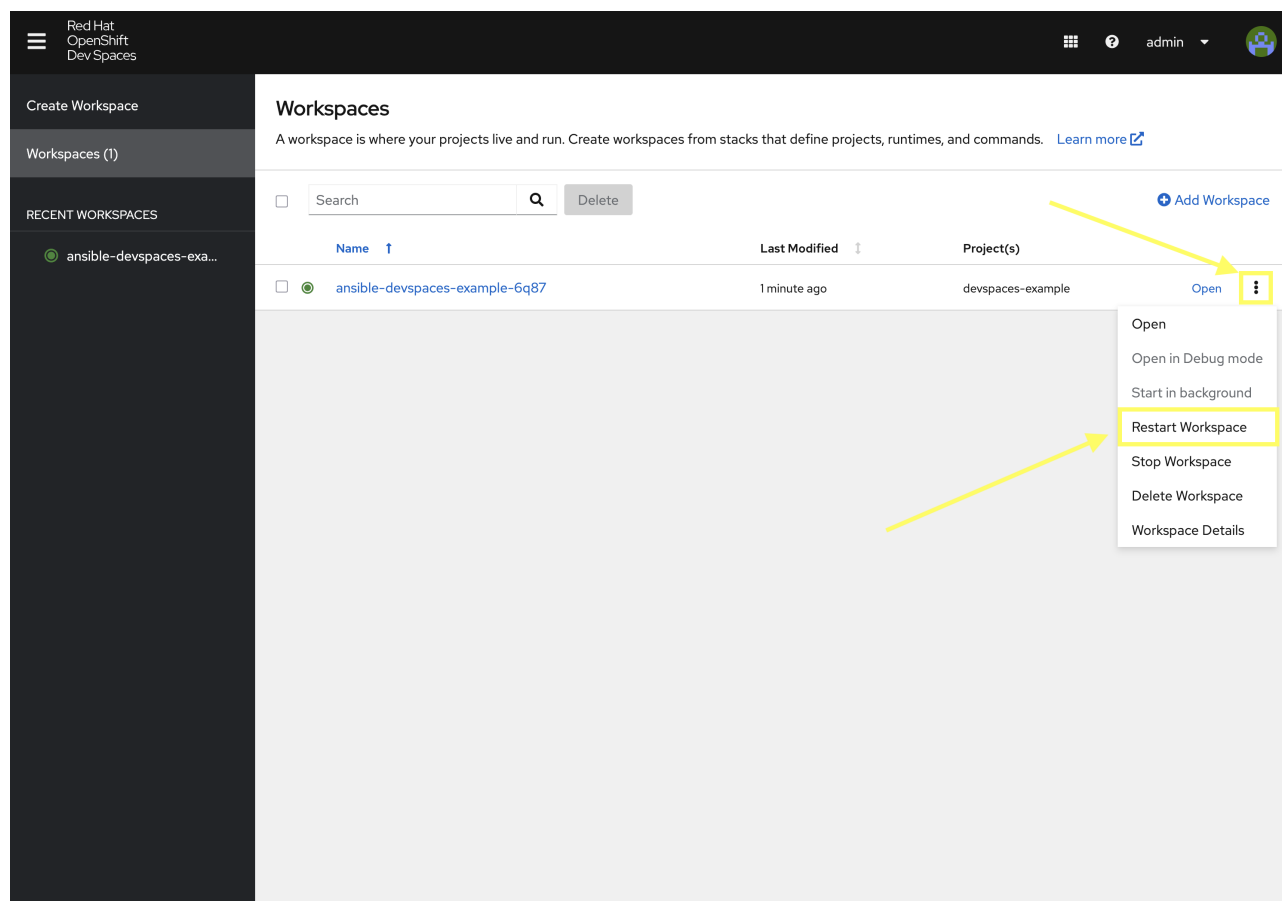
With the environment variables defined in the `ConfigMap` and `Secret` now defined within OpenShift, the next step is to restart your Dev Spaces workspace so that it can mount the values contained within these resources. To restart your Dev Spaces workspace, perform the following steps in the OpenShift web console:

1. Use the 9-block icon in the upper right to launch Red Hat OpenShift Dev Spaces directly from

the OpenShift console header.



2. Within the Dev Spaces console, on the *Workspaces* page, click the kebab menu (three vertical dots) next to your `ansible-dev-spaces-example-xxxx` workspace and select **Restart Workspace**.



3. Click the **Open** link next to your workspace to relaunch the workspace

When the workspace has started, you can verify that the environment variables are correctly mounted by executing the following command in the terminal which will check for the presence of the environment variables as defined in the **ConfigMap** and **Secret** resources:

```
env | grep -E "(AAP_MCP|LITELLM)"
```

The output should display values that correspond to the environment variables defined in the **ConfigMap** and **Secret** resources previously.

3.1.2: Configuring OpenCode

With the necessary environment variables now mounted into your Dev Spaces workspace, the next step is to introduce the external AI tool that will interact with Ansible Automation Platform via the Ansible MCP Server. For this section, we will utilize **OpenCode**, an open-source AI development platform that enables developers to build, deploy, and manage AI applications. OpenCode provides a unified interface for interacting with various AI models and services, making it easier to integrate AI capabilities into applications.

You may have noticed earlier that one of the environment variables defined in the **ConfigMap** resource was the **OPENCODE_CONFIG** property, which specifies the location of the OpenCode configuration file (Hint: you can run the command `env | grep OPENCODE_CONFIG` to see the value). This file does not exist yet as we will need to create it as part of this exercise. An **OpenCode configuration file** contains details related to the inference service as well as any MCP servers that should be utilized.

Create a new file named **opencode.json** at the within the **/projects/ansible-devspaces** directory containing the following:

/projects/ansible-devspaces/opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "autoupdate": false,
  "enabled_providers": [
    "LiteLLM"
  ],
  "mcp": {
    "aap": {
      "type": "remote",
      "enabled": true,
      "oauth": false,
      "url": "{env:AAP_MCP_URL}",
      "headers": {
        "Authorization": "Bearer {env:AAP_MCP_API_KEY}"
      }
    }
  },
}
```

```

"provider": {
  "LiteLLM": {
    "npm": "@ai-sdk/openai-compatible",
    "name": "LiteLLM",
    "options": {
      "baseUrl": "{env:LITELLM_URL}",
      "apiKey": "{env:LITELLM_API_KEY}"
    },
    "models": {
      "claude-sonnet-4-6": {
        "name": "{env:LITELLM_MODEL_NAME}",
        "limit": {
          "context": 35000,
          "output": 1000
        }
      }
    }
  }
}

```

In this configuration file, we are specifying details related to our inference service along with the AAP MCP Server. Take note of the variables beginning with `{env:…}` which will replace the values with the content injected from the *Secret* or *ConfigMap* defined previously.

3.1.3: Using OpenCode to Interact with the Ansible MCP Server

With all of the configuration steps complete, let's utilize OpenCode to interact with the Ansible MCP Server.

To launch OpenCode, execute the following command:

```
opencode
```

Like many other AI coding tools, OpenCode is a Text User Interface (TUI) based tool which allows you to interact directly with the tool. By launching OpenCode, you will be placed at a prompt for which you can enter commands or execute queries:

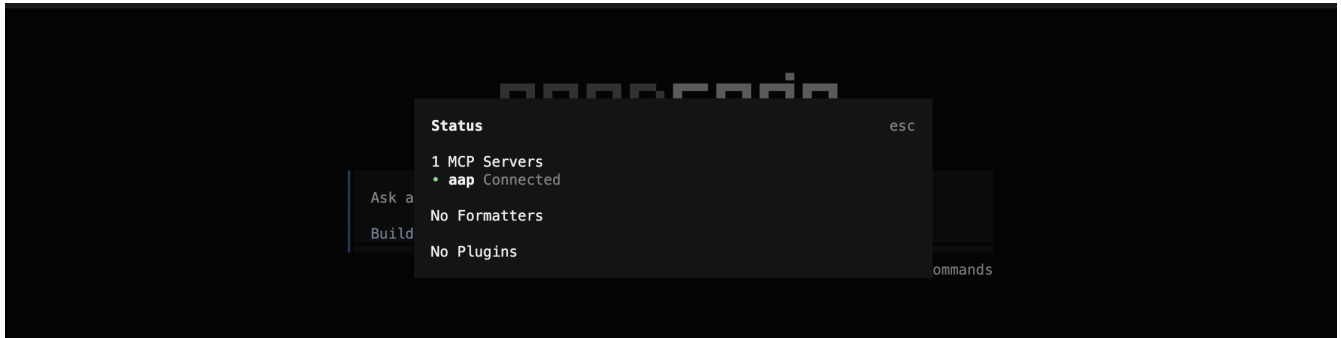


As you can see, OpenCode is using the LiteLLM inference service along with the `{litellm_available_models_list}` Model that was defined in the *ConfigMap*.

Use the `/status` command to view the status of the tool including any integration with external components, such as the AAP MCP server.

```
/status
```

If your MCP server is running and the API token that was stored in the *Secret* is being loaded properly, the status being displayed should look similar to the following:



The `aap` MCP server displays *Connected* indicating that it is able to integrate with Ansible Automation Platform as needed. If the status being displayed indicates an error, investigate any message/error code that may be present. In addition, you can view the logs from the AAP MCP Server pod that is running in the `aap` Project. The pod will have a name of `<instance_name>-mcp-xxxx` which may provide additional insights related to the failed integration.

Hit the `ESC` key to exit the status dialog to return to the main prompt.

Now, let's start a chat session using our natural language that retrieves details that originate from Ansible Automation Platform that is facilitated through the integration with the AAP MCP Server.

Let's ask for information related to the *Projects* that have been defined:

```
Display the names of aap projects
```

In a few moments, you will see OpenCode making queries to the *Tools* exposed by the AAP MCP Server and then providing a natural language response based on the information retrieved.

A successful invocation of the `project_list` MCP tool will display similar to the following:



Either the list of AAP project that are returned as a result of the `projects_list` tool invocation will be displayed or you can continue the conversation by asking additional questions to display more information related to the desired query.



Due to the non-deterministic nature of LLM's, the exact response provided by OpenCode may vary between invocations and may not match exactly what is

shown in the screenshot or steps above.

Feel free to continue the chat session or start a new session to continuing to converse with the LLM, obtaining realtime insights from Ansible Automation Platform though the use of the Ansible MCP Server.

Key Takeaways

- **MCP provides a standard way** to connect AI models with external systems.
- **Ansible MCP Server connects AI with AAP**, exposing AAP capabilities through MCP toolsets.
- **The MCP Server acts as the gatekeeper**, handling requests between the AI client and AAP.
- **Authentication and RBAC are enforced**, ensuring users can only access and perform actions permitted by their AAP permissions.
- **AAP data is standardized**, providing the AI with consistent and easier-to-process responses.
- **Natural-language interaction simplifies automation**, reducing the need for manual UI navigation or direct API calls.

Error Detection & AI-Powered Problem Solving with MCP

Welcome to Chapter 2! In Chapter 1, you learned **what** MCP is and **why** it matters. Now, you'll learn how MCP and Claude AI work together to solve Ansible playbook errors **automatically**.

We'll break this down into bite-sized sections that follow a simple pattern: **What is the problem?** → **How does MCP solve it?** → **What does this look like in practice?**

What You'll Learn

By the end of this chapter, you will be able to:

- Understand how Ansible playbooks fail and the typical troubleshooting process
- Explain how MCP enables automatic error detection and resolution
- Describe the complete workflow from error occurrence to fix application
- Understand how Claude AI analyzes errors and generates solutions
- Implement the operator approval workflow for safety and control
- Troubleshoot issues in the error resolution system

The Big Picture: Self-Healing Playbooks

Imagine a world where:

- Your Ansible playbook fails
- The system **automatically captures** the error details
- Claude AI **analyzes** the error in seconds
- An operator **reviews** Claude's suggested fix
- The fix is **applied automatically** with full safety checks
- Your playbook **retries** and succeeds

This is what we'll build in this chapter.

The Problem We're Solving

Current Situation (Without MCP)

When a playbook fails, operations teams typically:

1. **Get alerted** - Error message appears in logs
2. **Search for solution** - Google, Stack Overflow, docs (5-15 min)
3. **Diagnose the problem** - Check configs, variables, versions (10-20 min)

4. **Write a fix** - Update playbook or configuration (10-20 min)
5. **Test the fix** - Run playbook in test environment (10-20 min)
6. **Apply to production** - Deploy the fix carefully (5-15 min)
7. **Verify success** - Monitor for any new issues (5-10 min)

Total time: 1-2 hours per error (sometimes much longer for complex issues)

This is **reactive**, **manual**, and **inefficient**.

New Situation (With MCP + Claude)

With MCP and Claude:

1. **Error is captured** - Automatically, with full context (<5 sec)
2. **Claude analyzes** - AI examines error and generates fix (<30 sec)
3. **Operator reviews** - Human reviews Claude's solution (1-2 min)
4. **Fix is applied** - Automatically with backup created (<5 sec)
5. **Playbook retries** - Continues from failure point (varies)
6. **Success verified** - Automatic verification (<5 sec)

Total time: 2-5 minutes per error (including operator review)

This is **proactive**, **automated**, and **efficient**.

How It Works: The Five-Step Workflow

Step 1: Error Occurs

Your Ansible playbook runs and a task fails. The system immediately captures:

- Task name and which module failed
- Complete error message
- All variables and environment info
- Ansible version and logs
- Timestamp and affected hosts

Step 2: Error Sent to Claude via MCP

The error context is sent to Claude API through MCP. Claude receives structured information about:

- What failed and why
- The context in which it failed
- Recent configuration state
- Historical patterns (if available)

Step 3: Claude Analyzes & Provides Solution

Claude examines the error and provides: - **Root Cause Analysis** - Why did this fail? - **Explanation** - What does this error mean? - **Fixed Code** - Here's the corrected configuration - **Apply Steps** - Here's how to apply the fix - **Prevention Tips** - How to avoid this in future

Step 4: Operator Reviews & Approves

A human operator sees: - Clear explanation of the problem - Proposed fix with a diff showing changes - Risk assessment (will this break anything?) - Claude's confidence score - Then clicks "Approve" or "Reject"

Step 5: Fix Applied Automatically

Once approved, the system: - Creates a backup of original files - Validates the fix syntax - Applies the fix atomically - Retries the playbook - Logs everything for audit trail

Example: Real Error Resolution

Let's walk through a real example: **Docker image authentication error**.

The Error

```
FAILED - RETRYING: Deploy Docker container (Attempt 1 of 3)
fatal: [prod-1]: FAILED! => {
  "msg": "Failed to pull image: unauthorized: authentication required",
  "module_stderr": "",
  "module_stdout": "",
  "_ansible_parsed": true
}
```

What Happens Automatically

In less than 2 minutes:

```
[ERROR CAPTURE] Task failed: Deploy Docker container
[ERROR CAPTURE] Module: docker_image
[ERROR CAPTURE] Error: Failed to pull image: unauthorized
[ERROR CAPTURE] Host: prod-1
[ERROR CAPTURE] Gathering context...

[MCP SEND] Sending error to Claude API...

[CLAUDE ANALYSIS] Analyzing error...
[CLAUDE ANALYSIS] Root cause identified: Missing credentials
[CLAUDE ANALYSIS] Generating solution...
[CLAUDE ANALYSIS] Solution generated (confidence: 95%)
```

[OPERATOR REVIEW]

Claude's Analysis	
Root Cause: Docker registry auth	
Problem: Missing username/password	
Solution: Add registry credentials	
Proposed Change:	

```

- name: Deploy Docker container
  docker_image:
    name: myapp:latest
+   registry_url: docker.io
+   username: "{{ docker_user }}"
+   password: "{{ docker_pass }}"
    state: present

```

```

Risk: LOW
Approve? [y/n]: y

```

```

[FIX APPLY] Creating backup...
[FIX APPLY] Backup ID: bak_20260925_143022
[FIX APPLY] Applying fix...
[FIX APPLY] ✓ Fix applied successfully
[FIX APPLY] ✓ Retrying playbook...

[PLAYBOOK RETRY] Playbook running with fix...
[PLAYBOOK SUCCESS] ✓ All tasks completed successfully!

```

That's it! Error detected, analyzed, approved, fixed, and resolved in **2 minutes** with **full human oversight**.

Why MCP is Critical

Without MCP: * Claude would need manual description of error (no automation) * Operator would need to copy-paste error messages (error-prone) * No structured communication between systems (inefficient) * Each AI tool would need custom integration (duplicated work)

With MCP: * Error automatically structured and sent * All necessary context included automatically * Standard protocol works with any MCP-compatible tool * Claude has complete situational awareness * System can make informed decisions

Chapter 2 Structure

This chapter is organized as follows:

Section	Topic	Time
This Page	Introduction & big picture	10 min read
Section 1	How errors are detected	15 min read
Section 2	Claude integration via MCP	15 min read
Section 3	Operator approval workflow	15 min read
Lab	Hands-on error resolution	30 min lab

Prerequisites for This Chapter

To get the most from this chapter, you should:

- ☐ Have completed Chapter 1 (understand MCP basics)
- ☐ Understand basic Ansible playbook concepts
- ☐ Know what an Ansible error looks like
- ☐ Have access to AAP or test Ansible environment
- ☐ Have a Claude API key (from anthropic.com)

Don't have all of these? No problem! You can: * Read Chapter 1 first if you need MCP basics * Follow along conceptually without running commands * Review the provided example errors and solutions

What's Required to Succeed

For **reading/understanding**: * 2-3 hours to read all sections and examples * No special setup required

For **hands-on lab**: * Ansible 2.10+ installed locally or test environment * Access to AAP (Ansible Automation Platform) * Claude API key * ~1-2 hours to complete all exercises

Key Concepts You'll Learn

Error Capture - How system automatically detects and logs failures

Context Gathering - How to collect all information Claude needs

MCP Communication - How error details flow to Claude

AI Analysis - How Claude generates intelligent solutions

Approval Workflow - How operators maintain control and safety

Safe Application - How fixes are applied without risk

Ready to Begin?

Let's start with **Understanding How Playbooks Fail and How We Can Fix Them!**

→ **Next:** [Section 1: How Errors Are Detected](#)

Estimated reading time for Chapter 2: 60-90 minutes

Estimated lab time: 30-45 minutes

Mcp Server for playbook error detection and resolution.

This brief will demonstrate how the MCP server can be leveraged to identify errors in Ansible playbooks, use Claude to analyze the issues and recommend appropriate resolutions, and then automate the remediation process. To ensure the changes are applied in a controlled and governed manner, the Automation Orchestrator approval workflow can be integrated to provide a human-in-the-loop validation step before the fix is automatically implemented.

Lab Scenario: Hello World Sample with Error

We'll use a simple but intentional error to demonstrate the system:

```
---
- name: Hello World Sample
  hosts: all
  tasks:
    - name: Hello Message
      debug:
        msg: "Hello World"

    - name: APP version
      debug:
        msg: "{{ app_version }}"
```

we created a file `hello-world-playbook.yml` with the above content. The second task references a variable `app_version` that is not defined anywhere, which will trigger an error when the playbook is executed. This will allow us to observe how MCP captures the error and a prompt to Claude analyzes it.

Part 1: Configure GitHub in Ansible Automation Platform

Step 1.1: Access AAP Controller

1. Navigate to your AAP Controller URL from the RHDP login page.
2. Log in with your credentials.

Step 1.2: Create Project in AAP

1. Go to **Projects** (Resources → Projects)
2. Click **Create project**
3. Fill in the project details:

Field	Value
Name	MCP-Playbooks-Lab

Field	Value
Description	"Ansible Playbooks with MCP Error Detection"
Source Control Type	Git
Source Control URL	https://github.com/opentraining-labs-customizations/aap-playbooks.git

4. Click **Save**
5. Wait for the project to sync (you'll see a status message)

```
Status: Successful
Commit: abc1234567...
Branch: main
```

☐ **GitHub is now connected to AAP!**

Part 2: Create Job Template with MCP Integration

In AAP Controller:

1. Go to **Templates** (Resources → Templates)
2. Click **Create template** → **Job template**
3. Fill in the basic details:

Field	Value
Name	MCP-Error-Detection-Lab
Description	"Hello World playbook with MCP error detection"
Job Type	Run
Inventory	Demo-inventory
Project	MCP-Playbooks-Lab
Playbook	hello-world-playbook.yml

4. Click **Save**

☐ **Job Template is ready!**

Part 3: Run the Playbook

In AAP Controller:

1. Go to **Templates** (Resources → Templates)
2. Find **MCP-Error-Detection-Lab**
3. Click the **Launch** button (play icon)

Step 3.3: Monitor Job Execution

The job output will show:

```
PLAY [Hello World Sample] ****

TASK [Hello Message] ****
ok: [localhost] => {
    "msg": "Hello World"
}

TASK [APP version] ****
fatal: [localhost]: FAILED! => {
    "msg": "undefined variable: app_version"
}

PLAY RECAP ****
localhost: ok=1 changed=0 unreachable=0 failed=1

FAILED - RETRYING: APP version (Attempt 1 of 3)
```

❏ **Error detected! Now lets check with the MCP Server**

Part 5: Observe MCP Error Detection & Resolution

1. Go to Ansible devspace and open the vscode terminal and Run the command `opencode` to start the MCP server. You should see the following output:
2. Now give a Prompt to Claude to analyze the error. The system will automatically send the error context to Claude for analysis. Within 30 seconds, you will receive a proposed solution.
3. The output can be different from the following example, but it should look similar and the resolution would be to add the `aap_version`.
4. Now its upto the human to decide at what level of `playbook/ inventory/host_vars/group_vars` or extra variable the resolution should be proposed.

The Automation Orchestrator workflow can be integrated to provide a human-in-the-loop validation and proposed solution by the AI where with simple click of a button the Human can approve the fix and the system will automatically apply the fix and retry the playbook.

1. I will add the Variable to the Job Template extra variable section as shown below and click on the Save button and execute the Job Template again. The playbook should now succeed without any errors:

Part 6: Watch Playbook Retry and Succeed

Step 6.1: Playbook Retries with Fix

The playbook automatically retries:

```
PLAY [Hello World Sample] *****

TASK [Hello Message] *****
ok: [localhost] => {
  "msg": "Hello World"
}

TASK [APP version] *****
ok: [localhost] => {
  "msg": "1.0.0"
}

PLAY RECAP *****
localhost: ok=2 changed=0 unreachable=0 failed=0

✓ PLAYBOOK COMPLETED SUCCESSFULLY!
```

Step 6.2: Alternate fix option:

The applied fix (visible in job output or GitHub):

```
# Original (with error):
- name: APP version
  debug:
    msg: "{{ app_version }}"

# Fixed by Claude (auto-applied):
- name: APP version
  debug:
    msg: "{{ app_version | default('1.0.0') }}"

# Or with explicit variable:
vars:
  app_version: "1.0.0"
```

Step 6.3: Verify in GitHub

The changes are committed to GitHub:

```
git pull origin main
cat hello-world-playbook.yml
```

Output:

```
---
- name: Hello World Sample
  hosts: all
  vars:
    app_version: "1.0.0"
  tasks:
    - name: Hello Message
      debug:
        msg: "Hello World"

    - name: APP version
      debug:
        msg: "{{ app_version }}"
```

▣ **Playbook now has the fix applied!**

section2.adoc

Hands-On Exercises

Try these exercises to practice:

Exercise 1: Simple Query

Goal: Get a count of total jobs
Question: "How many jobs exist in my AAP?"
Expected: A number with job details
Learns: Basic job querying

Exercise 2: Filtered Query

Goal: Find specific jobs by status
Question: "Show me only successful jobs from the past week"
Expected: Filtered list of successful jobs
Learns: Query with conditions

Exercise 3: Multi-Step Query

Goal: Get multiple types of information
Question: "How many jobs failed this week and what were their error messages?"
Expected: List of failed jobs with error details
Learns: MCP can do multi-step queries

Exercise 4: Inventory Query

Goal: Understand your infrastructure
Question: "Show me all hosts grouped by operating system"
Expected: Breakdown of hosts by OS (Windows, Linux, etc)
Learns: Inventory management queries

Exercise 5: Follow-Up Conversation

Goal: Have a multi-turn conversation
Exchange:
You: "List my execution nodes"
AI: [Returns list of nodes]
You: "Which one has the most capacity?"
AI: [Analyzes and compares]
You: "What's the health status of that node?"
AI: [Queries monitoring data]
Learns: Contextual conversation

Part 7: Advanced Tips

Tip 1: Be Specific

Instead of:

"Show me jobs"

Say:

"Show me jobs that failed in the past 7 days with error details"

Specific queries get better answers.

Tip 2: Ask Follow-Up Questions

You: "What's the status of my platform?"
AI: [Gives health status]
You: "Which execution node is having issues?"
AI: [Digs deeper based on context]

The AI remembers context in the conversation.

Tip 3: Combine Information

You: "I have failed jobs and offline hosts. Are they related?"
AI: [Analyzes both, shows correlation]

The AI can synthesize information from multiple queries.

Tip 4: Use Natural Language

MCP isn't a CLI. You don't need exact syntax. Just ask naturally:

"How's my system doing?"
"Are my servers healthy?"
"What's been happening lately?"
"Do I have any problems?"

The AI will interpret and call the right tools.

Part 8: When to Use MCP vs Other Methods

Use MCP for:

- **Quick information** - "How many hosts do I have?"
- **Status checks** - "Is my platform healthy?"
- **Trend analysis** - "Have failures increased?"
- **Natural language** - Questions, not commands
- **Exploration** - "Show me everything about..."
- **Error Detection** - "Tell me why the sample-JT-name failed"

Use AAP Web UI for:

- **Detailed editing** - Configuring large projects
- **Visual dashboards** - Seeing graphs and charts
- **Bulk operations** - Multi-host changes
- **Training new users** - Learning the UI

Use AAP API for:

- **Automation** - Scripts and playbooks
- **Integration** - Custom tools
- **Exact control** - Precise parameter passing
- **Batch operations** - 1000s of changes

Summary Checklist

By now, you should have:

□ Run your first successful query against AAP □ Seen MCP tools being invoked □ Tried queries from multiple toolsets □ Had multi-turn conversations □ Encountered and solved at least one issue □ Understood what tools are available

Congratulations!

You've completed all three labs: 1. □ Deployed MCP on OCP 2. □ Configured your AI tool 3. □ Ran queries and practiced troubleshooting.

You now have a working MCP setup!

Appendix & Resources

MCP Cheat Sheet

Knowledge Check & Quiz

Troubleshooting Guide